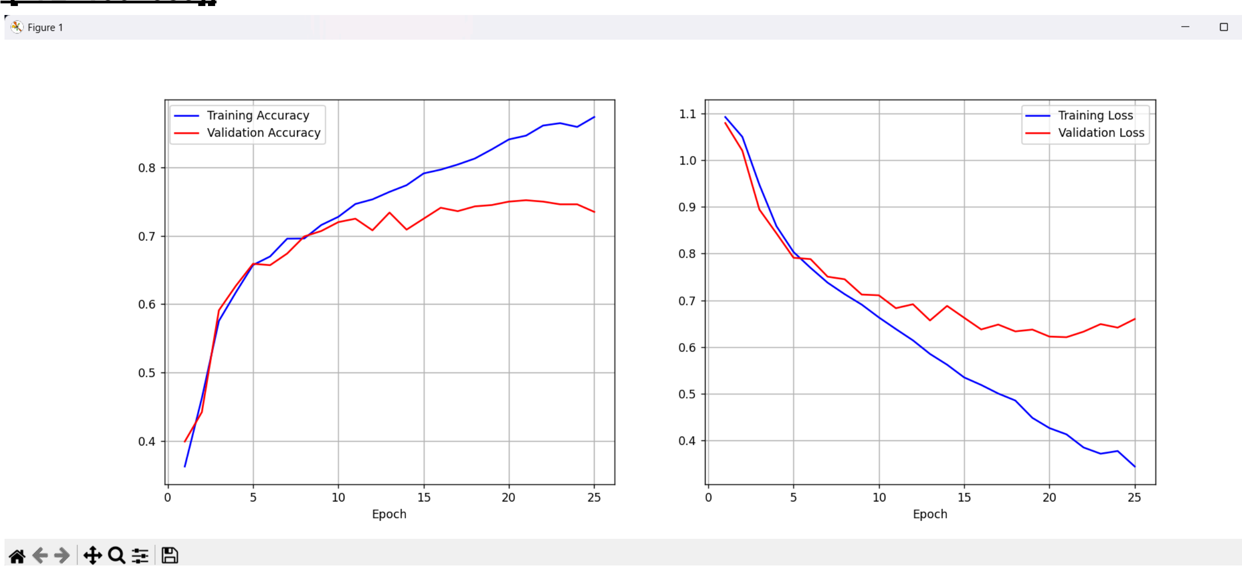
STEP 5

Hyper-parameter Optimization Summary
====================================
Best trial id: 1
Best validation accuracy: 0.7520
Best epoch (overfitting point proxy): 21
Test accuracy (best-val model): 0.7501

Best hyper-parameters:
num_conv_layers: 4
num_fc_layers: 1
dropout_count: 1
dropout_location: conv
dropout_rate: 0.2
learning_rate: 0.0001
max_epochs: 40

Confusion Matrix:
[[1301 336 137]
[197 919 117]
[72 100 659]]



Git commit of the project at this time:
https://github.com/BellaTheAlien/CMPM-146-P6/commit/4d8d3177f922ff965c621a2db52d01a34fd52ca0
First trying with 20 epochs in the run
Along with rescaling in basic model file from 150X150 down to 18x18, by layering 3 different times

STEP 6

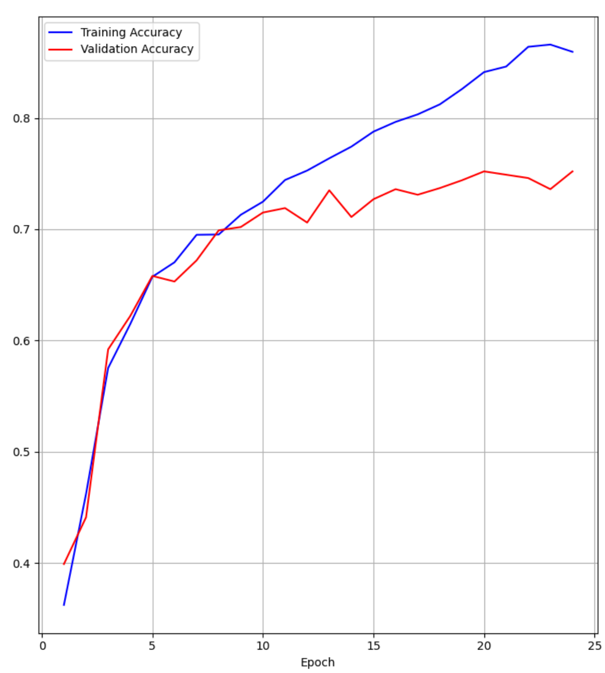
Hyper-parameter Optimization Summary

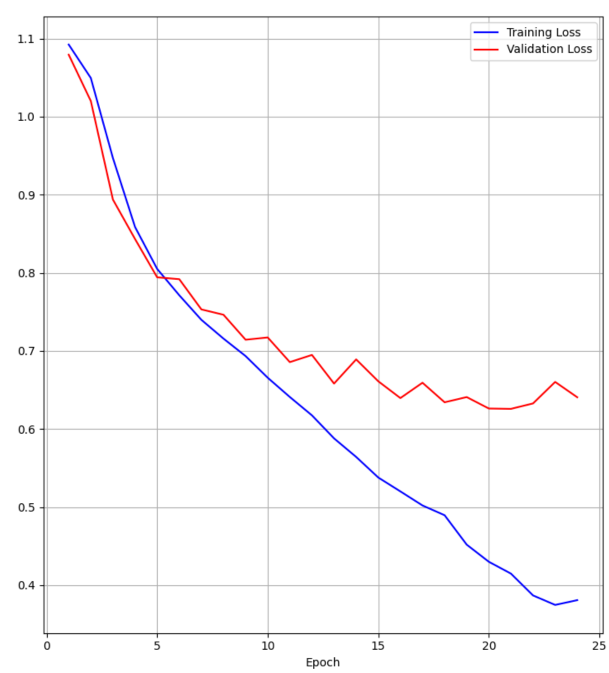
===================================
Best trial id: 1
Best validation accuracy: 0.7520
Best epoch (overfitting point proxy): 20
Test accuracy (best-val model): 0.7436

Best hyper-parameters:
  num_conv_layers: 4
  num_fc_layers: 1
  dropout_count: 1
  dropout_location: conv
  dropout_rate: 0.2
  learning_rate: 0.0001
  max_epochs: 40

Confusion Matrix:

```
[[1316  293  165]
 [ 218  876  139]
 [  80   89  662]]
```





Accuracy: 0.82 Validation Accuracy: 0.7440 – Loss: 0.4519
Search strategy optimization:

- Varied number of convolutional layers (2-4) → hierarchy depth
    - 2-3 layers → +2-5% accuracy
    - 3-4 layers → -1% to +1% accuracy because it can overfit or add noise
    - More layers help until capacity exceeds dataset complexity
    - Lower conv depths → underfit
- Varied number of fully connected layers (1-2)
    - More layers → increase overfitting
- Varied dropout layers (0-2), including location and rate
    - Dropout: 1 layer → rate: 0.2%
    - Light regularization was sufficient
    - Conv layer dropout helped generalization without destroying spatial features
    - Higher dropout or multiple dropout layers → underfitting
- Varied learning rate (1e-3, 5e-4, 3e-4)
    - Slower but more stable optimization
    - Avoided overshooting
- Used EarlyStopping on validation loss with restore_best_weights to select model near overfitting point
- Training over 20 epochs causes overfitting

To improve

- Add class weighted loss
- Increase dropout rate to 0.3

<u>STEP 7</u>

Tic-Tac-Toe Summary

```
===================================
| | | |
| | | |
| | | |
Turn: X
Player X took position (2, 1).
| | | |
| | | |
| |X| |
Turn: O
reference:
row 0 is neutral.
row 1 is happy.
row 2 is surprise.
```

2026-02-17 06:03:11.036787: I tensorflow/core/platform/cpu_feature_guard.cc:210] This TensorFlow binary is optimized to use available CPU instructions in performance-critical operations.
To enable the following instructions: SSE3 SSE4.1 SSE4.2 AVX AVX2 AVX512F AVX512_VNNI AVX512_BF16 AVX_VNNI FMA, in other operations, rebuild TensorFlow with the appropriate compiler flags.
Emotion detected as neutral (row 0). Enter 'text' to use text input instead (0, 1 or 2). Otherwise, press Enter to continue.

reference:
col 0 is neutral.
col 1 is happy.
col 2 is surprise.
Emotion detected as surprise (col 2). Enter 'text' to use text input instead (0, 1 or 2). Otherwise, press Enter to continue.

Player O took position (0, 2).
		O
	X	
Turn: X
Player X took position (2, 0).
		O
X	X	
Turn: O
reference:
row 0 is neutral.
row 1 is happy.
row 2 is surprise.
Emotion detected as happy (row 1). Enter 'text' to use text input instead (0, 1 or 2). Otherwise, press Enter to continue.

reference:
col 0 is neutral.
col 1 is happy.
col 2 is surprise.
Emotion detected as neutral (col 0). Enter 'text' to use text input instead (0, 1 or 2). Otherwise, press Enter to continue.

Player O took position (1, 0).
		O
O		
X	X	

Turn: X
Player X took position (2, 2).
		O
O		
X	X	X
Player X has won!

Questions:

- The interface worked reliably overall → being able to text override was useful
- It recognized the surprise facial recognition accuracy most of the time, however the model would switch up happy and neutral
    - The lighting could have confused the model because of the grayscale
    - Facial expressions are usually subtle rather than the exaggerated ones found in the dataset
    - Background can interfere with the facial recognition

Key lines of code added for emotional detection

Load trained model

```
if not hasattr(self, '_emotion_model'):
    model_paths = [
        'results/hpo_best_trial_1_baseline_1771332002.keras',
        'model.keras',
    ]
    loaded_model = None
    for path in model_paths:
        try:
            loaded_model = load_model(path)
            break
        except Exception:
            continue

    if loaded_model is None:
        raise FileNotFoundError('Cannot find a .keras model')

    self._emotion_model = loaded_model
```

Convert image to tensor and resize → convert grayscale to RBG

```
tensor_img = tf.convert_to_tensor(face_crop, dtype=tf.float32)

if len(tensor_img.shape) == 2:
    tensor_img = tf.expand_dims(tensor_img, axis=-1)
elif len(tensor_img.shape) != 3:
    raise ValueError('Unexpected image shape: {}'.format(tensor_img.shape))
```

```
resized = tf.image.resize(tensor_img, image_size)
if resized.shape[-1] == 1:
    resized_rgb = tf.image.grayscale_to_rgb(resized)
else:
    resized_rgb = resized[..., :3]
```

Add batch dimension

```
batch = tf.expand_dims(resized_rgb, axis=0)
```

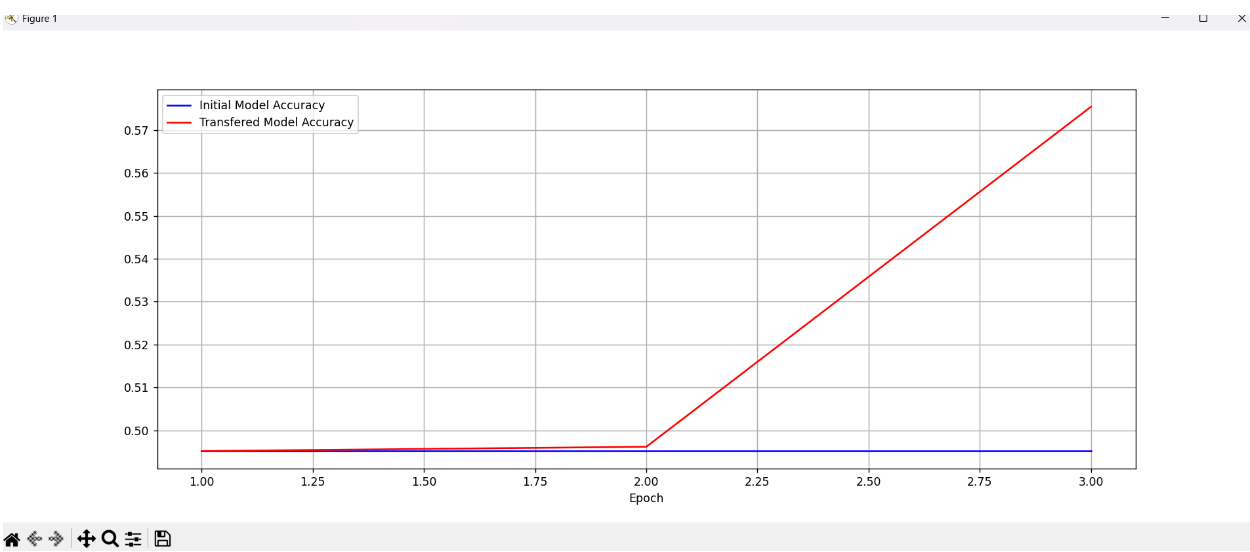
Run prediction

```
prediction = self._emotion_model.predict(batch, verbose=0)
predicted_idx = int(np.argmax(prediction, axis=-1)[0])
```

Map model label order to game label order

```
emotion = int(self._label_remap.get(predicted_idx, predicted_idx))
return emotion
```

STEP 8



Task: Dog vs Cat dataset used
Found 25000 files
Used 20000 files for training
Used 5000 files for validation
Total params: 9,494,048

Matrix analysis:
[[4000   0]
 [4000   0]]